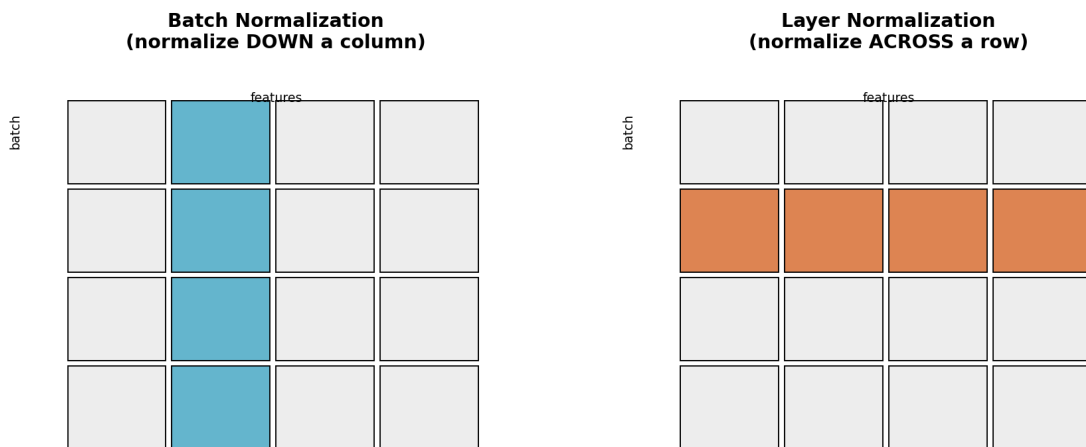


Layer Normalization vs Batch Normalization

in Transformers



Covers: What is normalization • Why we normalize • Benefits • Internal covariate shift • Batch normalization • Why self-attention doesn't use batch norm • How layer normalization works

1. What Is Normalization?

Normalization is the process of rescaling a set of values so they have a consistent, predictable distribution - typically mean 0 and standard deviation 1. In deep learning, this is applied to the activations flowing between layers, not just the raw input data.

What Is Normalization? Rescaling Values to a Consistent Range

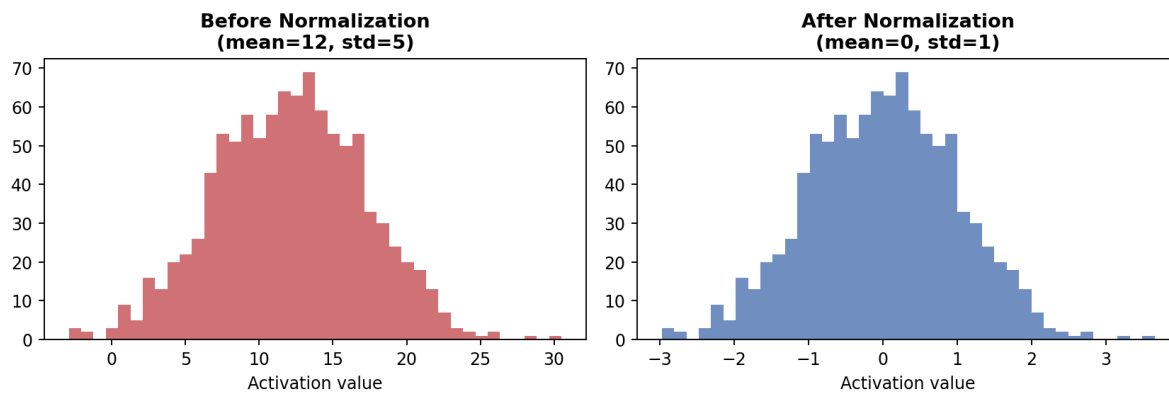


Figure 1: Normalization reshapes a wide, off-center distribution into a compact one centered at 0 with unit variance.

2. Why Do We Normalize?

As data flows through a deep network, activations can end up with wildly different scales at different layers - some very large, some very small, some centered far from zero. This makes optimization harder: gradients can vanish or explode, and the loss surface becomes bumpy and hard for gradient descent to navigate efficiently. Normalizing activations between layers keeps values in a stable, well-behaved range throughout training.

3. Internal Covariate Shift

This is the specific problem normalization was originally introduced to fix. As training proceeds and earlier layers' weights update, the **distribution of activations** feeding into later layers keeps changing - even though the network architecture itself hasn't changed. Each layer effectively has to keep re-learning how to handle a shifting input distribution, which slows down training considerably. This phenomenon is called **internal covariate shift**.

Internal Covariate Shift

As weights update during training, each layer's OUTPUT DISTRIBUTION keeps shifting - forcing every later layer to constantly re-adapt to a moving target

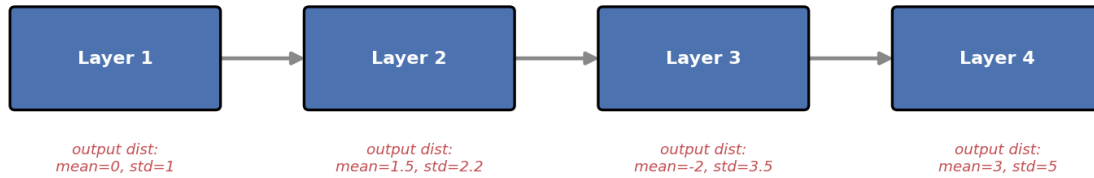


Figure 2: Without normalization, each layer's output distribution keeps drifting during training, forcing downstream layers to constantly adapt to a moving target.

Normalization stabilizes these intermediate distributions, giving every layer a more consistent input to work with, batch after batch, epoch after epoch.

4. Benefits of Normalization

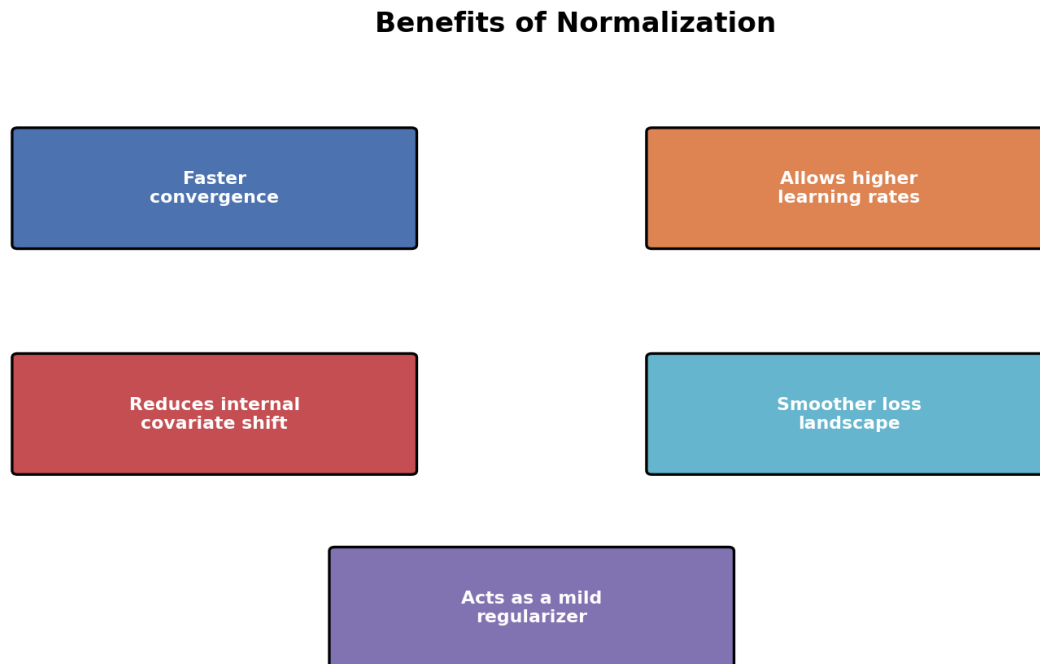


Figure 3: The practical training benefits normalization provides.

- **Faster convergence** - stable activation distributions mean gradient descent can make consistent, efficient progress instead of fighting a shifting landscape.
- **Allows higher learning rates** - without normalization, high learning rates often cause instability; with it, networks tolerate larger updates safely.
- **Reduces internal covariate shift** - the core problem described above.
- **Smoother loss landscape** - research has shown normalization makes the optimization surface less erratic, which is arguably even more important than the covariate-shift explanation alone.
- **Mild regularization effect** - the noise introduced by batch statistics (in batch norm specifically) can slightly reduce overfitting.

5. Batch Normalization

Batch Normalization (BatchNorm), introduced in 2015, normalizes activations using statistics computed **across the batch dimension** - for each individual feature, it computes the mean and variance across all examples currently in the mini-batch, then rescales using those statistics.

Batch Normalization - Normalizes ACROSS the Batch

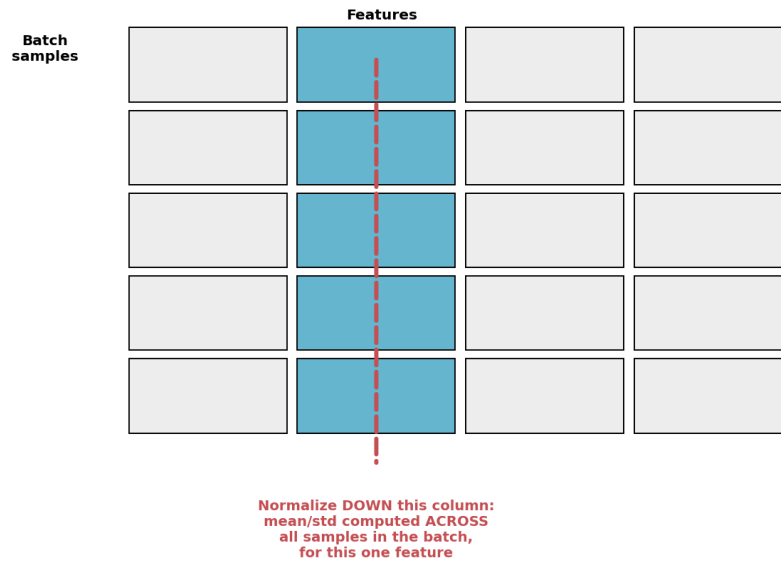


Figure 4: BatchNorm normalizes DOWN a column - using statistics computed across all samples in the batch, for one feature at a time.

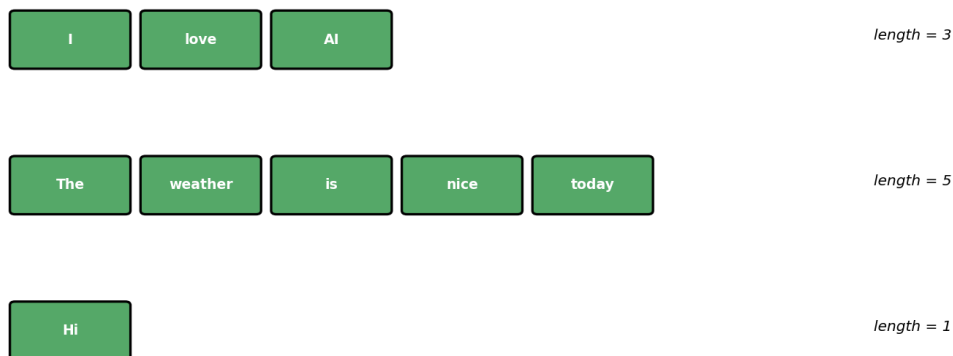
$$\text{BatchNorm}(x) = \gamma \cdot (x - \text{mean_batch}) / \sqrt{\text{var_batch} + \text{eps}} + \beta$$

Here, gamma and beta are learned scale/shift parameters that let the network recover the original representation power if pure normalization isn't ideal for a given layer. BatchNorm works extremely well for CNNs on fixed-size images with reasonably large batch sizes.

6. Why Don't We Use Batch Normalization in Self-Attention?

Batch Normalization runs into several specific problems when applied to Transformers and sequence data:

Why Batch Norm Doesn't Work Well for Self-Attention / Sequences



*Sequences in a batch have DIFFERENT lengths and lots of padding.
Computing statistics across the batch dimension mixes real tokens with padding,
and batch statistics become unstable/meaningless when batch size is small or variable.*

Figure 5: Sequences in a batch typically have very different lengths, requiring padding - which corrupts batch-wise statistics.

- **Variable sequence lengths:** text sequences in the same batch often have very different lengths and require padding. Computing statistics across the batch mixes real tokens with meaningless padding values, corrupting the normalization statistics.
- **Sensitive to batch size:** BatchNorm's statistics become noisy and unreliable with small batches - a common situation when working with long sequences or limited GPU memory.
- **Train/inference mismatch:** BatchNorm uses running averages of batch statistics at inference time (since there may be no "batch" at inference, e.g. generating one sequence at a time), which can behave inconsistently for sequence models compared to vision models.
- **Position-dependent statistics:** normalizing per feature across the batch treats every token position as interchangeable, which doesn't match how self-attention treats each token position as having its own distinct, evolving representation.

For all these reasons, the original Transformer paper (and virtually every Transformer since) uses **Layer Normalization** instead, which sidesteps every one of these issues.

7. How Does Layer Normalization Work?

Layer Normalization (LayerNorm) flips the axis BatchNorm uses. Instead of normalizing across the batch (for one feature at a time), it normalizes **across all features**, independently for each individual sample/token.

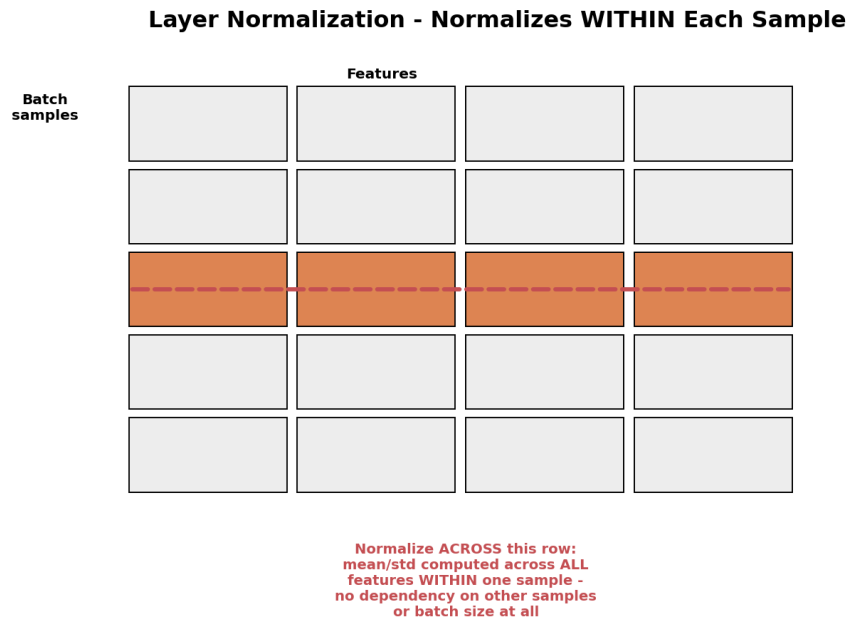


Figure 6: LayerNorm normalizes ACROSS a row - using statistics computed across all features of ONE sample, completely independent of the rest of the batch.

$$\text{LayerNorm}(x) = \gamma \cdot (x - \text{mean_features}) / \sqrt{\text{var_features} + \text{eps}} + \beta$$

Because the mean and variance are computed per-token (across its own embedding dimensions), LayerNorm has NO dependency on batch size, sequence length, or padding at all - each token normalizes itself, completely independently of every other token or sample. This makes it a perfect fit for sequence models with variable lengths and small or single-example batches.

Side-by-Side: The Core Difference

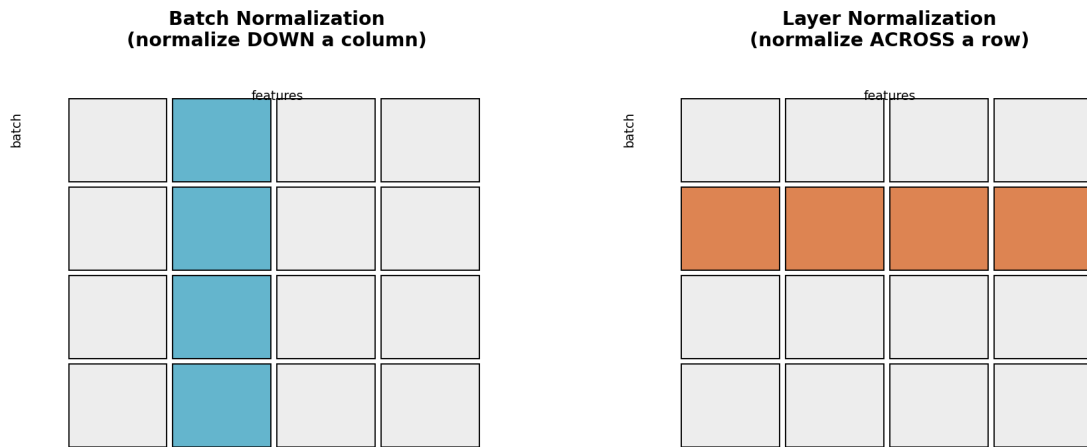


Figure 7: BatchNorm normalizes down a column (across the batch); LayerNorm normalizes across a row (across features, per sample).

Where LayerNorm Sits in a Transformer Block

LayerNorm appears twice in every Transformer block - once after the multi-head self-attention sub-layer, and once after the feed-forward sub-layer - each time paired with a residual (skip) connection.

Where LayerNorm Sits in a Transformer Block

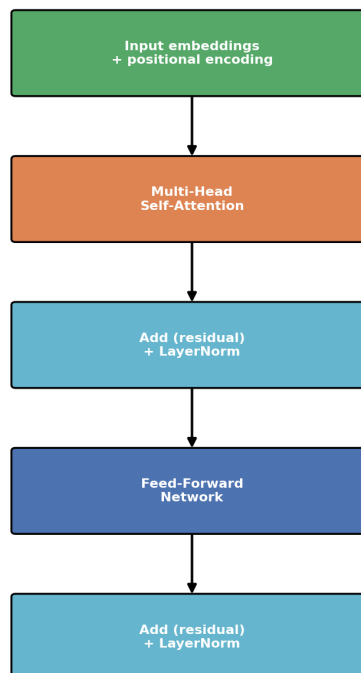


Figure 8: LayerNorm is applied after each residual addition, stabilizing activations before they pass to the next sub-layer.

Summary

Aspect	Batch Normalization	Layer Normalization
Normalizes across	The batch dimension (per feature)	The feature dimension (per sample)
Depends on batch size?	Yes - unstable with small batches	No - works even with batch size 1
Handles variable-length sequences?	Poorly - padding corrupts statistics	Yes - each token normalizes independently
Train vs inference behavior	Different (uses running averages at inference)	Identical at train and inference time
Common use case	CNNs on images, fixed-size batches	Transformers, RNNs, sequence models

Both techniques solve the same underlying problem - stabilizing activation distributions during training - but along different axes. BatchNorm's dependency on batch statistics makes it a poor fit for variable-length sequence data, which is exactly why every major Transformer architecture relies on LayerNorm instead.